

Randomized Neural Network with Petrov-Galerkin Methods for Solving Linear and Nonlinear Partial Differential Equations

Summary of Shang, Wang, and Sun (2023)

1 Introduction

This paper presents a novel numerical method called Randomized Neural Network with Petrov-Galerkin methods (RNN-PG) for solving partial differential equations (PDEs). The method combines the approximation power of neural networks with the mathematical foundation of the Petrov-Galerkin framework. RNN-PG addresses key limitations of both traditional numerical methods and existing deep learning approaches for PDEs by:

- Using randomized neural networks where only the weights of the output layer need to be trained, significantly reducing computational complexity
- Employing the Petrov-Galerkin framework with flexible test function spaces
- Using least-squares methods to solve the resulting linear system, avoiding the need to satisfy the discrete inf-sup condition
- Handling boundary conditions effectively without penalty methods
- Adopting a space-time approach for time-dependent problems

2 Algorithm Description

2.1 Problem Formulation

The RNN-PG method addresses PDEs of the form:

$$Au = f \quad \text{in } \Omega, \tag{1}$$

$$Bu = g \quad \text{on } \Gamma, \tag{2}$$

where Ω is a bounded domain in $\mathbb{R}^d$ with boundary $\Gamma = \Gamma_D \cup \Gamma_N$, $\Gamma_D \cap \Gamma_N = \emptyset$. The differential operator A can represent various PDEs, including diffusion-reaction equations:

$$A := -\nabla \cdot (\alpha(x)\nabla) + \delta(x), \quad (3)$$

where $\alpha_1 > \alpha(x) \geq \alpha_0 > 0$ and $\delta_1 \geq \delta(x) \geq \delta_0 > 0$. Boundary conditions can be Dirichlet, Neumann, or Robin types.

2.2 Petrov-Galerkin Framework

The weak formulation of the PDE is: Find $u \in U_{D,g_D} = \{u \in U; u|_{\Gamma_D} = g_D\}$ such that

$$a(u, v) = l(v) \quad \forall v \in V_{D,0} = \{v \in V; v|_{\Gamma_D} = 0\}, \quad (4)$$

where U and V are Hilbert spaces, and $a(\cdot, \cdot) : U \times V \rightarrow \mathbb{R}$ is a continuous bilinear form.

The traditional Petrov-Galerkin method approximates the solution by finding $u_h \in U_h \subset U_{D,g_D}$ such that

$$a(u_h, v_h) = l(v_h) \quad \forall v_h \in V_h \subset V_{D,0}, \quad (5)$$

where U_h and V_h are finite-dimensional function spaces, typically consisting of piecewise polynomial functions.

2.3 Randomized Neural Network Structure

The RNN-PG method replaces the trial function space U_h with a neural network space U_ρ . The neural network used has the form:

For a fully connected network with depth D :

$$\Phi_0(x) = x, \quad (6)$$

$$\Phi_l(x) = \rho(W_l \Phi_{l-1} + b_l) \quad \text{for } l = 1, \dots, D-1, \quad (7)$$

$$\Phi(x) = \Phi_D(x) = W_D \Phi_{D-1} + b_D, \quad (8)$$

where ρ is the activation function, W_l are weight matrices, and b_l are bias vectors.

The key innovation is that the weights and biases for all hidden layers ($l = 1, \dots, D-1$) are randomly initialized and then fixed. Only the weights of the output layer (W_D) are trainable parameters.

If Φ_j^{D-1} denotes the output of the j -th neuron in the $(D-1)$ layer, the neural network output is:

$$u_\rho(x) = \sum_{j=1}^{n_{D-1}} w_\rho^j \Phi_j^u(x), \quad (9)$$

where $\Phi_j^u := \Phi_j^{D-1}$ and $w_\rho^j := w_{1j}^D$. The degrees of freedom for the RNN-PG method is the number of neurons in the $(D-1)$ layer, i.e., n_{D-1} .

2.4 RNN-PG Algorithm

The RNN-PG method solves: Find $u_\rho \in U_\rho \subset N_\rho(D, M_D, B_D) \subset U$ such that

$$a(u_\rho, v_h) = l(v_h) \quad \forall v_h \in V_h \subset V_{D,0}, \quad (10)$$

$$u_\rho(x_k) = g_D(x_k) \quad \text{for points } x_k \in \Gamma_D, k = 1, 2, \dots, N_b, \quad (11)$$

The algorithm can be summarized as follows:

1. Initialize neural network architecture $u_\rho : \Omega \rightarrow \mathbb{R}$ with depth D
2. Fix the parameters of all layers except the last layer and rewrite u_ρ as a linear combination of basis functions
3. Choose test functions $v_i \in V_h$ and assemble the linear system $AU = L$
4. Sample random points on the Dirichlet boundary and impose boundary conditions, leading to $BU = G$
5. Solve the least-squares problem with the combined linear system $\begin{bmatrix} A \\ B \end{bmatrix} U = \begin{bmatrix} L \\ G \end{bmatrix}$
6. Update the output layer parameters of u_ρ

2.5 Mixed RNN-PG Methods

The paper also introduces Mixed RNN-PG (M-RNN-PG) methods that approximate multiple unknown functions simultaneously. For example, for the Poisson equation, the mixed formulation introduces an auxiliary variable $p = \nabla u$, resulting in a first-order system:

$$p - \nabla u = 0 \quad \text{in } \Omega, \quad (12)$$

$$-\nabla \cdot p = f \quad \text{in } \Omega. \quad (13)$$

Different mixed formulations are presented, each with specific advantages. For implementation, each unknown function (u and p) is approximated with its own neural network, and the resulting system is solved using the least-squares method.

2.6 Space-Time RNN-PG for Time-Dependent Problems

For time-dependent problems, the RNN-PG method employs a space-time approach, treating temporal and spatial variables jointly and equally. For

example, for the heat equation:

$$\frac{\partial u}{\partial t} - \nabla \cdot (\alpha(x) \nabla u) = f \quad \text{in } \Omega \times I, \quad (14)$$

$$u(x, t) = g_D(x, t) \quad \text{on } \Gamma_D \times I, \quad (15)$$

$$\alpha(x) \nabla u(x, t) \cdot n = g_N(x, t) \quad \text{on } \Gamma_N \times I, \quad (16)$$

$$u(x, 0) = h_0(x) \quad \text{in } \Omega, \quad (17)$$

where $I = (0, T)$ is the time interval.

The RNN-PG method treats this as a higher-dimensional problem in $\Omega \times I$. The initial condition becomes another boundary condition on $\Omega \times \{0\}$. The weak formulation and neural network construction are similar to the stationary case, but with input dimension $n_0 = d + 1$ to account for both spatial and temporal variables.

For nonlinear time-dependent problems like Burger's equation, the method employs nonlinear iteration techniques such as Picard iteration or Newton's method in conjunction with the space-time approach.

3 Theoretical Findings

3.1 Universal Approximation Properties

The paper leverages the theoretical foundation that neural networks with continuous, non-polynomial activation functions can uniformly approximate any continuous function on a compact domain. Specifically, the work references the following theorem:

Given $p \geq 1$, $s, k, d \in \mathbb{N}^+$, $s \geq k + 1$. Let ρ be the logistic function $\frac{1}{1+e^{-x}}$ or tanh function $\frac{e^x - e^{-x}}{e^x + e^{-x}}$. Denote $F_{s,p,d} := \{f \in W^{s,p}([0, 1]^d) : \|f\|_{W^{s,p}([0, 1]^d)} \leq 1\}$. For any $\epsilon > 0$ and $f \in F_{s,p,d}$, there exists a neural network $f_\rho \in N_\rho(D, M_D, B_D)$ with depth $D \leq C \log(d + s)$, $M_D \leq C \cdot \epsilon^{-d/s-k-\mu_k}$, and $B_D \leq C \cdot \epsilon^{-\theta}$ such that

$$\|f - f_\rho\|_{W^{k,p}([0, 1]^d)} \leq \epsilon, \quad (18)$$

where C, θ are constants depending on d, s, p, k ; μ is an arbitrarily small positive number.

This theorem establishes that PDE solutions can be efficiently approximated by neural networks with an adequate number of layers and nonzero weights.

3.2 Independence from Inf-Sup Condition

A significant theoretical advantage of the RNN-PG method is its independence from the inf-sup condition. Traditional Petrov-Galerkin methods require careful design of the finite element pair U_h and V_h to satisfy the discrete

inf-sup condition:

$$\inf_{u_h \in U_h} \sup_{v_h \in V_h} \frac{a(u_h, v_h)}{\|u_h\|_U \|v_h\|_V} = \inf_{v_h \in V_h} \sup_{u_h \in U_h} \frac{a(u_h, v_h)}{\|u_h\|_U \|v_h\|_V} = \beta_h > 0, \quad (19)$$

By using the least-squares method to solve the resulting linear system, the RNN-PG method avoids this constraint, making it more flexible in the choice of test functions. This is particularly advantageous for the Mixed RNN-PG method, where traditional approaches would require carefully chosen function spaces to satisfy the LBB condition.

3.3 Error Dependency

The paper establishes that the error between the neural network solution u_ρ and the exact solution u depends on:

- The approximation ability of the function space U_ρ
- The choice of test function space V_h
- The collocation points on the Dirichlet boundary

Numerical experiments demonstrate that the accuracy of the RNN-PG method improves with:

- Larger number of neurons in the hidden layers (DoF)
- More test functions (finer mesh)
- Deeper neural networks
- More collocation points for boundary conditions

4 Numerical Examples and Results

The paper demonstrates the efficacy of the RNN-PG method through several numerical examples:

4.1 Poisson Equation

A 2D Poisson equation with mixed boundary conditions and a smooth solution $u = \cos(\pi x) \sin(\pi y)$ was solved. The RNN-PG method achieved very high accuracy with a small number of degrees of freedom:

- With mesh size $h = 2^{-5}$ and DoF = 200, the relative L^2 error was 9.651×10^{-10} and the H^1 error was 5.167×10^{-9}
- Compared to finite element methods (FEM), RNN-PG achieved significantly better accuracy with far fewer degrees of freedom

- A cubic FEM (P_3) with 9215 DoF had errors of 1.495×10^{-7} and 1.125×10^{-5} , while RNN-PG with only 200 DoF achieved better accuracy

The authors also compared the performance of RNN-PG with different types of test functions (piecewise bilinear functions, neural networks, Legendre polynomials, and Chebyshev polynomials), showing that piecewise bilinear functions resulted in better conditioning of the linear system.

4.2 Heat Equation

A 2D heat equation with an exact solution $u(x, y, t) = 2e^{-t} \sin(\pi x/2) \sin(\pi y/2)$ was solved using the space-time approach. The RNN-PG method treated this as a 3D problem (2D spatial + 1D temporal) and achieved excellent results:

- With mesh size $h = 2^{-4}$ and DoF = 800, the L^2 error at time $t = 1$ was 9.499×10^{-10} and the H^1 error was 1.635×10^{-8}
- Compared to the combination of P_2 FEM for spatial discretization and backward Euler for temporal discretization, RNN-PG achieved much higher accuracy with fewer computational resources
- The RNN-PG method required no time-stepping, avoiding error accumulation issues

4.3 Wave Equation

A 2D wave equation with exact solution $u(x, y, t) = \sin(\pi x/2) \sin(\pi y/2) \sin(\pi t/2)$ was solved. The space-time RNN-PG method achieved excellent results:

- With mesh size $h = 2^{-4}$ and DoF = 800, the L^2 error at time $t = 1$ was 1.395×10^{-9} and the H^1 error was 4.394×10^{-8}

4.4 Advection-Diffusion Equation

A 1D advection-diffusion equation was treated as a 2D space-time problem. The RNN-PG method outperformed the hp-VPINN method (Variational Physics-Informed Neural Networks with hp-refinement):

- RNN-PG with DoF = 400 achieved higher accuracy than hp-VPINN with 1296 training parameters
- RNN-PG satisfied boundary and initial conditions exactly without penalty parameters

4.5 Burger’s Equation

A 2D nonlinear Burger’s equation with exact solution $u(x, y, t) = 1/(1 + e^{(x+y-t)/(2\epsilon)})$ was solved for different values of ϵ :

- For $\epsilon = 1$, $h = 2^{-3}$, DoF = 400, the RNN-PG method achieved L^2 error of 1.335×10^{-8} and H^1 error of 1.012×10^{-7} at time $t = 1$ after three Newton iterations
- For $\epsilon = 0.1$ (sharper solution profiles), the method still performed well with L^2 error of 5.193×10^{-3}
- Compared to PINNs, RNN-PG was significantly more accurate and required fewer iterations

4.6 High-Dimensional Heat Equation

A high-dimensional heat equation with exact solution $u = \|x\|_d^2 + 2t$ was solved for dimensions $d = 5, 10, 20$:

- For $d = 5$ with DoF = 3200, the L^2 error was 8.529×10^{-7} with runtime of 10.28 seconds
- For $d = 10$ with DoF = 3200, the L^2 error was 2.710×10^{-6} with runtime of 12.42 seconds
- For $d = 20$ with DoF = 3200, the L^2 error was 5.396×10^{-5} with runtime of 15.50 seconds
- The method outperformed PINNs in both efficiency and accuracy

5 Implications and Advantages

The RNN-PG method offers several significant advantages over both traditional numerical methods and existing neural network-based approaches:

5.1 Computational Efficiency

- By only training the output layer weights, RNN-PG dramatically reduces the number of trainable parameters
- The method converts the PDE solution process into solving a linear system, avoiding the difficulties of training deep neural networks with optimization methods
- For time-dependent problems, the space-time approach eliminates the need for time-stepping, reducing computational cost and avoiding error accumulation

- The method shows nearly linear scaling with the number of collocation points and training parameters
- Runtime for high-dimensional problems is significantly faster than other methods (e.g., solving a 20D problem in just 15.5 seconds)

5.2 Accuracy and Approximation Power

- RNN-PG achieves extremely high accuracy with very few degrees of freedom compared to traditional FEM
- The method maintains high accuracy even for problems with complex solutions and high dimensions
- For the examples tested, the method achieves errors on the order of 10^{-8} to 10^{-10} for many problems, far superior to what is typically achieved with PINNs
- Neural networks provide excellent approximation power for smooth solutions, and the RNN-PG method effectively leverages this property

5.3 Flexibility in Implementation

- The method is mesh-free, eliminating the need for complex mesh generation
- It handles different boundary conditions easily within the same framework
- The choice of test functions is flexible, allowing for adaptation to problem characteristics
- Mixed formulations can be implemented straightforwardly without concerns about the inf-sup condition
- The method can be applied to both linear and nonlinear problems, stationary and time-dependent

5.4 Scalability to High Dimensions

- RNN-PG effectively addresses the "curse of dimensionality" that limits traditional PDE solvers
- The method maintains good performance even for 20-dimensional problems
- By using randomized neural networks and the space-time approach, the method remains computationally tractable for high-dimensional problems

5.5 Potential for Future Development

The paper identifies several areas for further development:

- Designing randomized neural networks to better capture features of exact solutions
- Using domain decomposition with different neural networks in different subdomains
- Developing preconditioners to improve the conditioning of the linear system
- Implementing adaptive refinement strategies
- Parallelizing the algorithm for large-scale problems